

101 AI Basics for Developers

A practical workshop for using IDE agents without losing engineering judgment.

VS Code + GitHub Copilot framing

30 minutes

30 minutes | VS Code + GitHub Copilot mental model



LLMs do three things: read pieces, use context, make a best guess.

1

Pieces

The model reads tokens: little chunks of words, code, punctuation, and symbols.

2

Context

It only knows what is in front of it right now: your prompt, selected code, tool results, and prior messages.

3

Best guess

It predicts a likely answer. Useful can still be wrong.



Your job

Let it draft, then ground the answer in code, tests, docs, and judgment.

Fast draft. Incomplete context. Always verify the important parts.

A good prompt tells the agent five simple things.

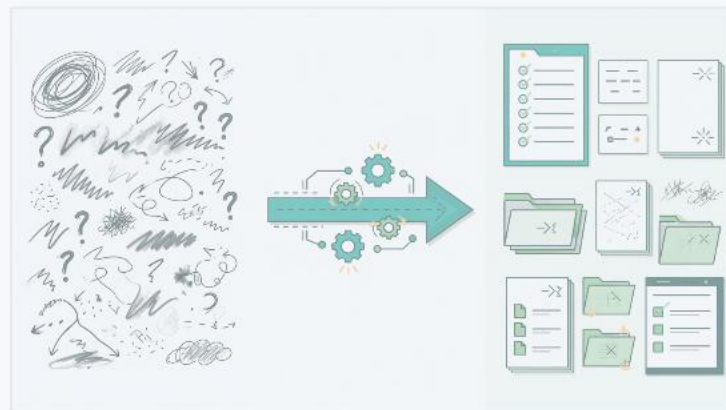
Weak prompt

Fix auth bug.

→ Too much room to guess

Agent-ready prompt

Find why expired sessions still pass /api/me. Start in auth middleware and session tests. Make the smallest fix. Update tests. Summarize what changed.



A prompt is just a clearer ticket: what to do, where to start, and how we know it worked.

What to fix

Where to look

What to avoid

How to check

Done

Copilot does not carry the whole repo. It packs a working context.

1

What you point at

Opened files, selected code, issue text, logs, and instructions.

2

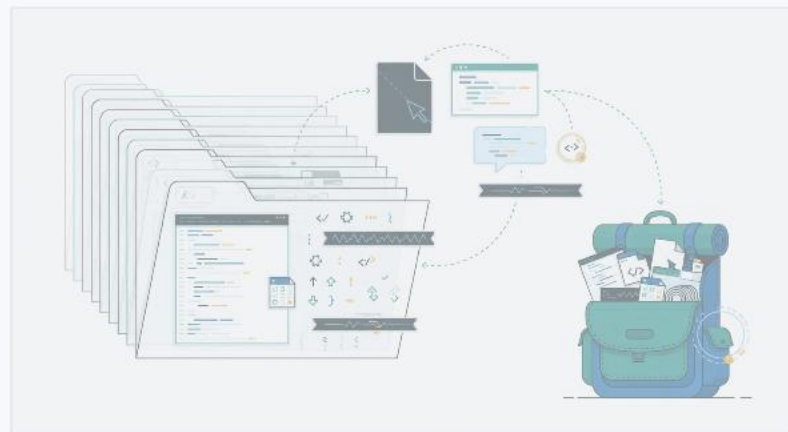
What it searches for

Related files, symbols, tests, docs, and matching patterns.

3

What tools return

Search results, test output, diffs, errors, and prior steps.



If the right file never gets packed, the answer can sound right and still miss your repo.

Ask for the map before edits: files inspected, tests found, assumptions, and risk.

Do not give it everything. Give it the right starter packet.

Better context is not more stuff. It is the few facts that change the next decision.

The ask

What should change, and what should stay alone.

The evidence

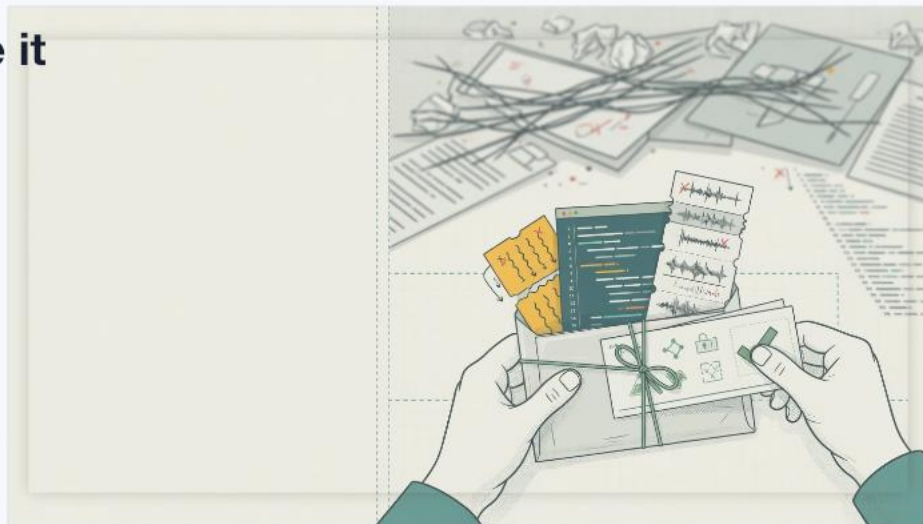
Exact files, failing logs, screenshots, errors, or issue text.

The rules

Repo conventions, risky areas, generated files, API boundaries.

The check

Tests to run, behavior to prove, and what counts as done.

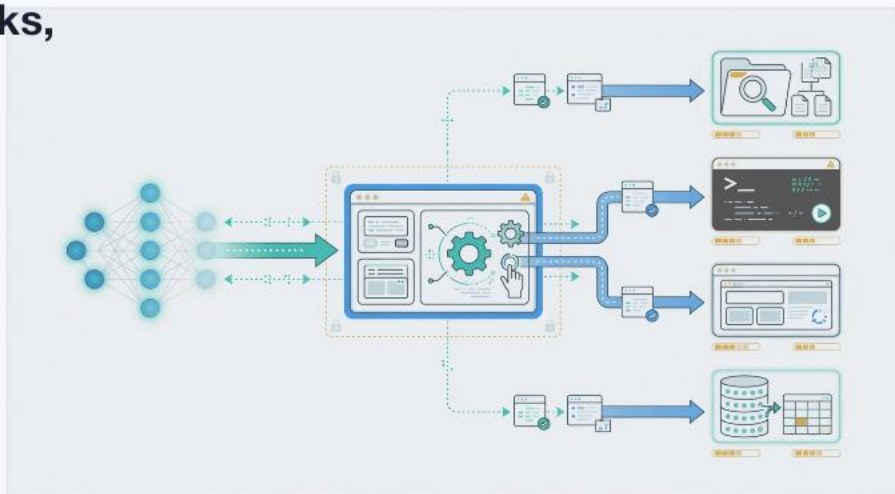


Start narrow. Add context when the agent proves it needs more.

Tool calling means: the model asks, the app runs, results come back.

The model is not running tests in its imagination. The host runs the command and feeds back the output.

Tools have blast radius



Trust tool results more than vibes. Still check permissions, side effects, and weird failures.

Retrieval finds candidates. You still decide what is true.

Retrieval is orientation, not proof.

Search by meaning

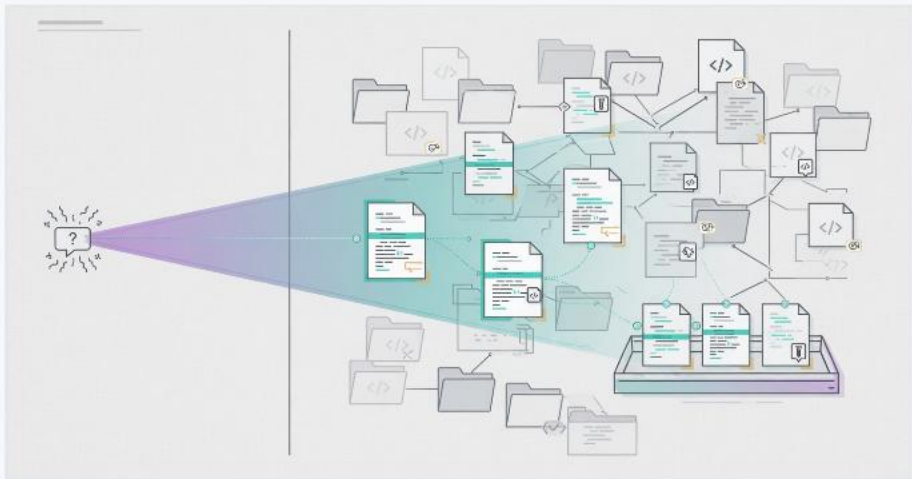
Good when you do not know the exact file, symbol, or doc name.

Treat results as leads

A close match can still be stale, nearby, or from the wrong path.

Read the source

Open cited files, callers, tests, configs, and docs before trusting it.



Ask the agent to separate found evidence from inferred conclusions.

MCP is a standard plug for agent tools. It is not automatic trust.

MCP standardizes the shape of tool access. Your security model still decides who gets the keys.

Client

The agent app needs a capability.

Server

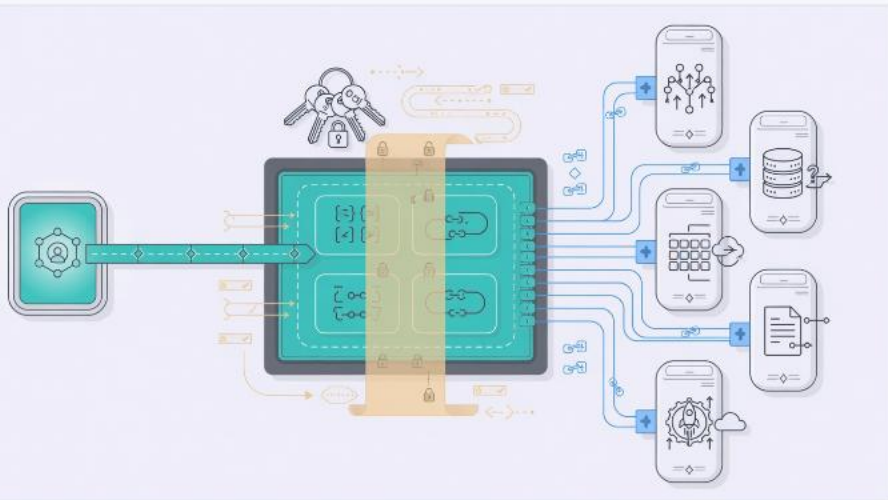
A tool server describes names, schemas, resources, and results.

Host

The host decides what is allowed and what needs approval.

Systems

Git, docs, data, tickets, deploys, calendars, and more.



Good MCP setup means clear tools, narrow permissions, logs, and review for risky actions.

Use a workflow when the path is known. Use an agent when the path must be found.

Do not call it an agent just because it sounds cooler.

Workflow

Known path

Same steps each time. Easier to test, audit, and run cheaply.

Agent

Discovered path

Explores files, tools, logs, and hypotheses as the work unfolds.



Is the path repeatable?

Does discovery matter?

What needs review?

Repeatability favors workflows. Ambiguity favors agents. Both still need verification.

Choose the mode that keeps review possible.

More autonomy is not better. It is just more blast radius.

Ask

Understand

Explain, compare, locate, summarize. No edits.

Edit

Targeted change

Known files, small surface, focused review.

Agent

Explore + execute

Search, edit, run commands, test, and iterate.

Plan

Map before code

Use when scope is foggy or blast radius is high.

Scope clear?

Tests clear?

Easy to revert?

Increase autonomy only when the verification story improves with it.



Give agents work you can review in one sitting.

If you cannot review the diff, you did not delegate a task.

Good fit test

Small surface

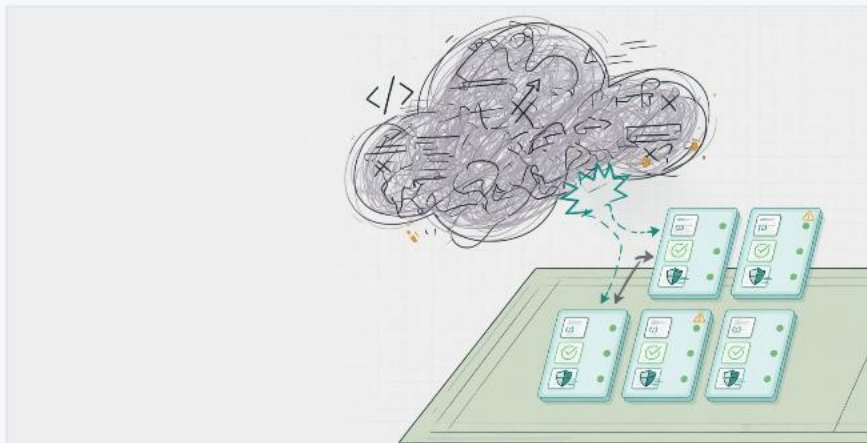
One feature, bug, route, component, or test area.

Clear check

A test, log, screenshot, or manual behavior to verify.

Readable diff

A human can inspect the change without a scavenger hunt.



Split these first

Make it production ready

Fix all flaky tests

Rewrite the architecture

01

One
behavior

02

One area

03

One
verification

Big goal -> small inspectable task -> agent session -> human review.

Stay in the loop: frame, watch, interrupt, verify.

You are still the engineer. The agent is the worker, not the reviewer.

Interrupt when you see:

Wrong files

No tests

Surprise scope

Uncited claims



01

Frame

Goal, scope,
starting files,
constraints.

02

Watch

Plan, files read,
tool calls,
assumptions.

03

Interrupt

Stop drift while it
is still cheap.

04

Verify

Diff, tests,
behavior, residual
risk.

Steering early keeps reviews small and sessions trustworthy.

Confidence is not a check. Evidence is.

Most failures come from missing context, loose scope, or weak verification.

Ask before you trust it:

What evidence?

What changed?

What was verified?

What risk remains?

Invented API

Ask for the local symbol or docs.

Wide diff

Constrain files and public APIs.

Stale context

Restart, summarize, or checkpoint.

Weak check

Run focused tests or manual QA.

Risky action

Gate commands, secrets, and deploys.



Make the agent show evidence, limits, checks, and leftover risk.

Write down the rules the agent should not guess.

Repo instructions are an operating manual, not a policy novel.

Repo rules

Package manager, generated files, safe commands, style decisions.

Path rules

Tests, APIs, frontend patterns, infra, migrations, security.

Workflow recipes

Bug triage, review prep, release notes, incident checks.

Keep out

Secrets, risky commands, deploys, policy guesses, stale preferences.



Good places for durable rules:

Copilot instructions

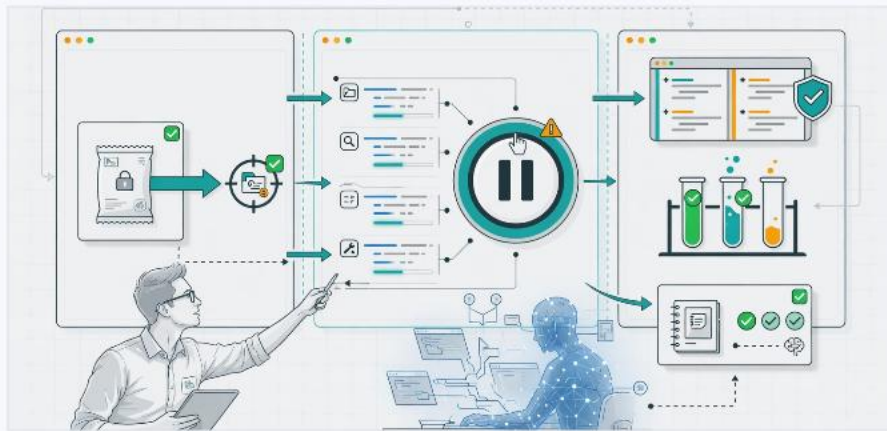
AGENTS.md

Skills / prompts

Short, durable instructions make every future session less guessy.

Use one checklist: before, during, after.

Do not wait until the end to become the engineer.



Before

Shape the task

- ☐ Define done
- ☐ Bound the scope
- ☐ Name the checks

During

Watch direction

- ☐ Read the plan
- ☐ Watch files and tools
- ☐ Interrupt drift

After

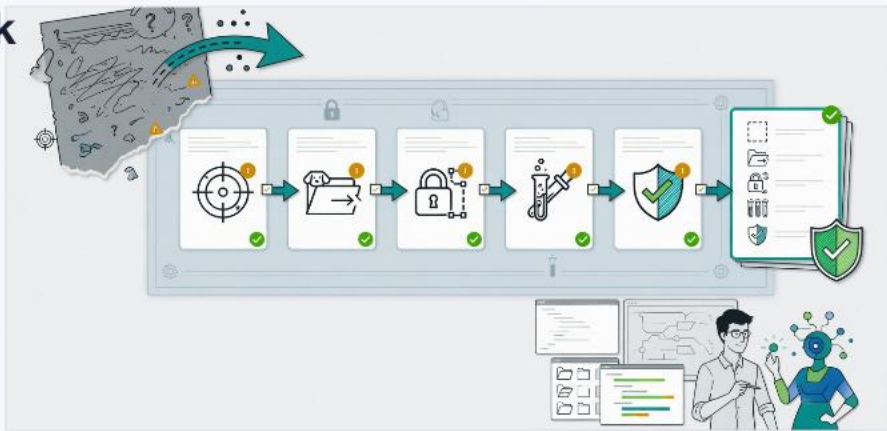
Prove it worked

- ☐ Inspect the diff
- ☐ Run tests or QA
- ☐ Capture learning

Before gives direction. During catches drift. After proves the work.

Rewrite vague work into a task you can verify.

The prompt is the work ticket. Make it bounded and checkable.



Vague ask

Make the settings page better.



Agent-ready task

Improve settings-page accessibility and spacing only. Start in SettingsPage. Do not change routes or API behavior. Add focused tests. Summarize before/after and remaining risk.

Good agent work is bounded, testable, and reviewed.